Doğrudan Bellek Erişimi (DMA) ve Gömülü Sistemlerde Bellek Yönetimi Teknik Eğitim Raporu

1. Giriş ve Temel Kavram Tanımları

Gömülü sistem tasarımında yüksek veri aktarım hızlarına ulaşırken sistem kaynaklarını ve enerji bütçesini verimli kullanmak en kritik mühendislik hedeflerinden biridir. Bu hedefe ulaşmak için öncelikle işlemci ve bellek mimarisini oluşturan temel bileşenlerin işlevlerini doğru tanımlamak gerekir.

- **CPU [Central Processing Unit - Merkezi İşlem Birimi - Program komutlarını yürüten ve mantıksal kararları alan ana işlemci birimi]:** Sistemdeki komutları sırasıyla çalıştıran, aritmetik ve mantıksal işlemleri gerçekleştiren ana denetleyicidir.
- **RAM [Random Access Memory - Rastgele Erişimli Bellek - Verilerin geçici olarak tutulduğu çalışma belleği]:** Çalışma zamanında verilerin, değişkenlerin ve tamponların geçici olarak saklandığı yüksek hızlı bellek alanıdır.
- **Peripheral (Çevre Birimi):** Mikrodenetleyici içinde veya dışında yer alan, dış dünya ile iletişim kuran (örneğin seri haberleşme veya analog sinyal dönüştürme) özel donanım bloklarıdır.
- **Register (Yazmaç):** Çevre birimleri veya CPU içinde yer alan, donanımın durumunu kontrol etmeye, yapılandırmaya ve anlık verileri tutmaya yarayan çok hızlı bellek hücreleridir.
- **Bus (Veri Yolu):** İşlemci, bellek ve çevre birimleri arasında veri, adres ve kontrol sinyallerini taşıyan ortak iletken hat altyapısıdır.
- **Buffer (Tampon):** Donanım aktarım hızı ile yazılımın veriyi işleme hızı arasındaki zaman uyumsuzluklarını dengelemek amacıyla RAM'de ayrılan geçici veri biriktirme alanıdır.

DMA Kavramı ve Genel Görevi

DMA [Direct Memory Access - Doğrudan Bellek Erişimi - İşlemci müdahalesi olmadan bellek ile çevre birimleri arasında yüksek hızlı veri aktarımı sağlayan donanım birimi], ana işlemci olan CPU'yu her bir veri öğesini tek tek okuyup yazma yükünden kurtaran özel bir bus master (veri yolu yöneticisi) donanım birimidir.

DMA'nın temel görevi; desteklenen bellek bölgeleri ile çevre birimleri arasında veya bellekten belleğe veri bloklarını yüksek hızla taşımaktır. DMA, veri kopyalama ve taşıma işlemlerini üstlenerek CPU üzerindeki işlem yükünü hafifletir; böylece CPU'nun diğer karmaşık uygulama hesaplamalarını yürütmesine veya düşük güç modlarında (sleep mode) kalarak enerji tasarrufu sağlamasına olanak tanır.

Pedagojik Önemli Not: DMA Sihirli Bir Donanım Değildir

Mühendislik öğrencilerinin en sık düştüğü yanılgı, DMA'nın veriyi "sihirli bir şekilde, sıfır maliyetle" taşıdığı ve sistemin genel kararlarını kendisinin aldığı varsayımıdır.

DMA genel uygulama programını çalıştırmaz; gelen bir verinin anlamını analiz edemez, kararlar veremez veya hangi mesaja yanıt verileceğini seçemez. Karar verme ve genel uygulama mantığı tamamen CPU'nun sorumluluğundadır. CPU; DMA'yı yapılandırır, başlatır ve aktarım bittiğinde oluşan kesme olaylarını yönetir. Ayrıca DMA aktarımı maliyetsiz değildir. DMA bir veri yolu yöneticisi olarak fiziksel veri yollarını (bus) ve bellek bant genişliğini tüketir. Bir aktarım sırasında CPU ile aynı veri yoluna veya bellek bloğuna erişmek istediğinde veri yolu çakışmalarına (bus contention) ve bellek erişim gecikmelerine neden olur.

---

2. CPU ile Veri Kopyalama ve DMA Mimarisi Karşılaştırması

Geleneksel gömülü yazılım mimarilerinde veri aktarımı Polling (Sorgulama) veya Interrupt (Kesme) yöntemleriyle CPU üzerinden yürütülür. DMA tabanlı aktarım ise bu süreçleri donanımsal boyuta taşır.

Aktarım Yöntemlerinin Karşılaştırması

Aktarım Yöntemi	Aktarım Adımları	Veri Yolu Kullanımı	CPU Müdahalesi ve Yanıt Süresi
Polling (Sorgulama)	1. Çevre birimi bayrağını oku. 2. Veri hazır olana kadar bekle. 3. Yazmaçtan veriyi oku. 4. RAM adresini hesapla ve yaz. 5. Sayacı düşür.	CPU tüm döngü boyunca veri yolunu meşgul eder; sürekli okuma komutları çalıştırır.	**Tam Müdahale:** CPU aktarım bitene kadar kilitlenir (blocking). Gecikme süresi yüksektir, verimsizdir.
Interrupt (Kesme)	1. Veri hazır olduğunda kesme oluşur. 2. CPU bağlamı saklar (context switch). 3. **ISR [Interrupt Service Routine - Kesme Hizmet Rutini - Kesme oluştuğunda CPU tarafından çalıştırılan özel donanım sürücü fonksiyonu]** içinde yazmaç okunur ve RAM'e yazılır. 4. ISR'den çıkılır ve bağlam geri yüklenir.	Kesme anında veri yolu kısa süreliğine CPU komut okuma ve yazma adımları için kullanılır.	**Yüksek Müdahale:** Her bayt/kelime için CPU kesmeye girer. Bağlam kaydetme/yükleme (overhead) işlemciyi yorar.
DMA Tabanlı Aktarım	1. CPU, DMA'yı yapılandırır ve başlatır. 2. Donanım isteğiyle DMA veriyi doğrudan RAM'e taşır. 3. Blok bitince tek bir bitiş kesmesi üretilir.	DMA ve CPU veri yolunu ortaklaşa (arbitration ile) paylaşır. Sadece veri taşıma vuruşunda bus kullanılır.	**Minimum Müdahale:** CPU sadece başlangıç yapılandırmasını yapar ve blok bitiminde haberdar edilir. Yanıt süresi optimizedir.

CPU Tabanlı Kopyalama Adımları

Kesme veya sorgulama tabanlı geleneksel kopyalamada CPU şu adımları her veri öğesi için döngüsel olarak çalıştırır:

1. Çevre birimi veri yazmacındaki (data register) hazır durumunu kontrol eder.
2. Yazmaç adresindeki veri okunarak CPU dahili yazmaçlarına (register file) yüklenir.
3. Hedef **SRAM [Static Random Access Memory - Statik Rastgele Erişimli Bellek - Mikrodenetleyici içindeki yüksek hızlı sistem RAM'i]** adresi hesaplanır.
4. CPU dahili yazmacındaki veri hedef SRAM adresine yazılır.
5. Aktarım sayacı bir azaltılır, bellek adresi artırılır ve sayaç 0 olana kadar işlem tekrarlanır.

DMA Tabanlı Aktarım ve Veri Yolu Yetkilendirmesi (Bus Arbitration)

DMA tabanlı aktarımda ise süreç çevre biriminin doğrudan DMA denetleyicisine bir istek (DMA Request) göndermesiyle başlar. DMA denetleyicisi sistem veri yolunu (Bus) kullanmak için hakemlik birimine başvurur. Veri yolu kontrolünü devraldığında, veriyi kaynak adresten hedef adrese doğrudan aktarır, sayaç ve adres güncellemelerini donanımsal olarak yapar.

Aynı anda hem CPU hem de DMA denetleyicisi aynı RAM bloğuna veya veri yoluna erişmek isteyebilir. Bu durumda **Hakemlik (Arbiter)** mekanizması devreye girer. Hakemlik birimi, tanımlanan öncelik seviyelerine (**Priority**) göre veri yolunun kullanım hakkını belirli çevrimler için birimlere tahsis eder. Böylece sistemde kilitlenme (starvation) engellenir.

---

3. DMA Aktarım Parametreleri ve Konfigürasyon Mimarisi

Bir DMA aktarımının başlatılabilmesi için donanım yazmaçlarının yazılım tarafından doğru parametrelerle yapılandırılması gerekir.

3.1 Adresleme ve Yön

- **Kaynak Adresi (Source Address):** Taşınacak verinin ilk okunacağı fiziksel bellek veya çevre birimi yazmaç adresidir.
- **Hedef Adresi (Destination Address):** Verinin yazılacağı bellek alanı veya çevre birimi yazmaç adresidir.
- **Aktarım Yönü (Direction):** Aktarımın yönünü belirler; Çevre Biriminden Belleğe (Peripheral-to-Memory), Bellekten Çevre Birimine (Memory-to-Peripheral) veya Bellekten Belleğe (Memory-to-Memory) olabilir.

3.2 Veri Genişliği ve Sayacı

- **Veri Erişim Genişliği (Data Width / Beat Size):** Tek bir bus erişiminde (beat) taşınacak veri boyutudur. Genellikle 8-bit (Byte), 16-bit (Half-Word) veya 32-bit (Word) olarak ayarlanır.
- **Aktarım Adedi (Transfer Counter / Block Transfer Count):** Bir DMA bloğunda toplam kaç adet veri erişimi (beat) yapılacağını belirleyen 16-bit veya 32-bitlik sayaç değeridir.
- **Adres Artış Kuralları (Increment / Fixed):** Her aktarım vuruşundan sonra adreslerin otomatik olarak artırılıp (Increment) artırılmayacağını (Fixed) belirler. Örneğin, bir RAM

tamponuna veri yazılırken hedef adresi artırılır; sabit bir alıcı/verici veri yazmacından okuma yapılırken kaynak adresi sabit tutulur.

3.3 Tetikleme ve Kaynaklar

DMA aktarımının başlamasını sağlayan **İstek/Tetik (Request/Trigger)** kaynakları üç grupta toplanır:

1. **Yazılım Tetiklemesi (Software Trigger):** Doğrudan yazılım komutuyla başlatılır (Genellikle Bellekten Belleğe aktarımlarda kullanılır).
2. **Çevre Birimi Tetiklemesi (Peripheral Trigger):** Bir çevre biriminin verisinin hazır olmasıyla üretilir (Örn: ADC dönüşümünün bitmesi veya UART alıcı tamponuna veri gelmesi).
3. **Olay Sistemi Tetiklemesi (Event System Trigger):** Mikrodenetleyici içindeki bağımsız olay veri yolu (Event System) üzerinden gelen donanımsal sinyallerle başlatılır.

3.4 Kanal, Akış ve Tanımlayıcı Yapıları

- **Kanal (Channel):** Belirli bir çevre biriminin veya yazılım isteğinin DMA denetleyicisine bağlandığı donanımsal istek yoludur.
- **Akış (Stream):** Bazı gelişmiş mikrodenetleyicilerde (örneğin STM32 mimarisinde) esnek çevre birimi eşlemesine ve bağımsız FIFO tamponlarına sahip esnek DMA aktarım yollarıdır.
- **Tanımlayıcı (Descriptor / Transfer Control Descriptor):** Aktarım parametrelerini (kaynak, hedef, sayaç, kontrol bitleri) içeren veri yapısıdır.

Gelişmiş DMA yapılarında tanımlayıcılar SRAM içinde bağlı liste (**Linked-list**) şeklinde saklanabilir. Böylece bir aktarım bittiğinde DMA denetleyicisi CPU müdahalesi olmadan SRAM'deki sonraki tanımlayıcı adresini (**Descriptor Next Address**) otomatik okuyarak ardışık ve karmaşık aktarımları kesintisiz yürütebilir.

---

4. DMA Aktarım Yönleri ve Tampon Yönetim Modları

Aktarım Yönleri

1. **Çevre Biriminden Belleğe (Peripheral-to-Memory):** Sensörlerden, ADC'den veya haberleşme birimlerinden (UART/SPI) gelen verilerin RAM tamponlarına aktarılması. (Kaynak adresi sabit, hedef adresi artan).
2. **Bellekten Çevre Birimine (Memory-to-Peripheral):** RAM'deki verilerin (örneğin DAC veya UART verisi) çevre birimlerine iletilmesi. (Kaynak adresi artan, hedef adresi sabit).
3. **Bellekten Belleğe (Memory-to-Memory - M2M):** RAM içindeki bir diziden başka bir diziye yüksek hızlı veri kopyalama. (Hem kaynak hem hedef adresi artan, genellikle yazılım tetiklemeli).

Modlar ve Tampon (Buffer) Stratejileri

Normal Mod (Normal Mode)

Yapılandırılan aktarım sayısı (Transfer Counter) sıfıra ulaştığında DMA aktarımı durur. Yeniden aktarım yapmak için CPU'nun kanalı tekrar yapılandırması ve etkinleştirmesi gerekir.

Döngüsel Mod (Circular Mode)

Sayaç sıfırlandığında ve tamponun son adresine ulaşıldığında DMA denetleyicisi otomatik olarak başlangıç adresine ve başlangıç sayaç değerine geri döner. Aktarım kesintisiz devam eder.

- **Pedagojik Uyarı:** Döngüsel mod sistem bellek kapasitesini artırmaz. Eski veriler işlenmediyse yeni gelen veriler eski verilerin üzerine yazılır (**Overwrite**).

Çift Tampon Yapısı (Double-Buffer / Ping-Pong Buffer) Zaman Çizelgesi

İki ayrı bellek tamponunun veya tek bir büyük tamponun iki yarısının kullanıldığı stratejidir. DMA denetleyicisi ilk yarıyı doldururken CPU ikinci yarıyı işler.

Adım Adım Zaman Çizelgesi (0 ms - 200 ms):

- **t = 0 ms:** DMA aktarımı başlar. DMA, Tampon A'yı (**0 - 99**. örnekler) doldurmaya başlar. Sahiplik: Tampon A → DMA.
- **t = 100 ms:** Tampon A dolar. DMA, Yarım Aktarım Kesmesi (**Half-Transfer Interrupt**) üretir. DMA otomatik olarak Tampon B'yi (**100 - 199**. örnekler) doldurmaya geçer. CPU kesmeyi alır; Tampon A'nın sahipliği CPU'ya geçer. CPU **t = 100 ms** ile **t = 200 ms** penceresinde Tampon A'yı işler.
- **t = 200 ms:** Tampon B dolar. DMA, Tam Aktarım Kesmesi (**Full-Transfer Interrupt**) üretir. DMA tekrar Tampon A'ya döner. Tampon B'nin sahipliği CPU'ya geçer. CPU Tampon B'deki veriyi işler.

```
Tampon Sahipliği ve Zaman Akışı:
[ t = 0 ms -> 100 ms ]   : DMA -> [Tampon A (Dolar)]   | CPU -> [Beklemede/Diğer İşler]
[ t = 100 ms -> 200 ms ] : DMA -> [Tampon B (Dolar)]   | CPU -> [Tampon A'yı İşliyor]
(Half-Transfer IT)
[ t = 200 ms -> 300 ms ] : DMA -> [Tampon A (Dolar)]   | CPU -> [Tampon B'yi İşliyor]
(Full-Transfer IT)
```

Olaylar ve Kesmeler

- **Yarım Aktarım Kesmesi (Half-Transfer Interrupt):** Tamponun ilk yarısı dolduğunda üretilen kesmedir. CPU'ya ilk yarıdaki veriyi işleme fırsatı verir.
- **Tam Aktarım Kesmesi (Full-Transfer Interrupt):** Tamponun tamamı dolduğunda üretilen kesmedir. CPU'ya ikinci yarıyı işleme fırsatı verir.

Eşzamanlılık Riski ve Durum Yönetimi

İki bağımsız yürütücünün (CPU ve DMA) aynı bellek alanına erişmesi eşzamanlılık riskleri doğurur:

- **Tampon Sahipliği (Buffer Ownership):** Belirli bir anda tamponun hangi bölümünü kimin kontrol ettiğinin belirgin olması kuralıdır.

- **Yarış Koşulu (Race Condition):** CPU veri okurken DMA'nın aynı adrese yeni veri yazması durumudur.
- **Üstüne Yazma (Overrun / Overwrite):** CPU'nun veriyi işleme hızının DMA'nın veri doldurma hızından yavaş kalması sonucu işlenmemiş verinin kaybolmasıdır.

Yazılım bu durumları engellemek için durum bayrakları (Boş, Dolduruluyor, Hazır, İşleniyor) ve kritik bölüm korumaları kullanmalıdır.

---

5. Detaylı Sayısal Uygulama Örneği: ADC Veri Toplama Sistemi

Bu bölümde, **ADC [Analog-to-Digital Converter - Analog-Dijital Dönüştürücü - Analog voltaj seviyelerini sayısal verilere dönüştüren çevre birimi]** veri toplama senaryosu adım adım sayısal olarak çözümlenmiştir.

Girdi Verileri ve Varsayımlar

- **Sinyal Örnekleme Hızı: 1000 örnek/saniye (1 kHz)**
- **Örnek Başına Veri Boyutu:** 16-bit (2 bayt) (Örnek verinin 16-bitlik alanda saklandığı varsayılmıştır).
- **Hedef Tampon Kapasitesi:** 100 adet 16-bit örnek
- **Timer (Zamanlayıcı) Tetikleme Periyodu: 1 ms** (**1 / 1000 s**)

Adım Adım Hesaplama ve Analiz

1. Toplam Bayt Miktarının Hesaplanması

Tek bir blokta toplanacak toplam veri boyutu:

Toplam Bayt = Örnek Sayısı × Öğe Genişliği (Bayt)

Toplam Bayt = 100 örnek × 2 bayt/örnek = 200 Bayt

2. Tampon Dolma Süresinin Hesaplanması

Bir bloğun (100 örnek) tamamlanması için geçen süre:

Süre = (Örnek Sayısı)/(Örnekleme Hızı)

Süre = (100)/(1000 örnek/s) = 0.1 s = 100 ms

Bu hesaplama, her **1 ms**'de bir timer tetiklemesiyle kararlı bir şekilde 1 örnek üretildiği varsayımına dayanır. ADC'nin örnekleme zamanlamasını DMA kendi kendine belirlemez; zamanlayıcı (timer) ve ADC tetikleme düzeni belirler.

3. Çift Tampon (Ping-Pong) Bellek Analizi ve CPU İşleme Penceresi

Sistemin kesintisiz çalışması için her biri 100 örnek içeren iki yarıdan (Ping-Pong) oluşan tek bir döngüsel tampon tanımlansın:

- **Toplam Örnek Sayısı: 100 örnek (Yarı 1) + 100 örnek (Yarı 2) = 200 örnek**
- **Toplam Bellek İhtiyacı: 200 örnek × 2 bayt = 400 Bayt**

```
Bellek Düzeni (400 Bayt Toplam):
[ Yarı 1: 200 Bayt (0 - 99. örnekler) ] [ Yarı 2: 200 Bayt (100 - 199. örnekler) ]
  |-- 0 - 100 ms arası dolar              |-- 100 - 200 ms arası dolar
  |-- Half-Transfer IT üretir             |-- Full-Transfer IT üretir
```

CPU İşleme Penceresi Analizi:

- İlk **100 ms** sonunda Yarım Aktarım Kesmesi (**Half-Transfer IT**) oluşur. CPU'nun ilk 200 baytlık veriyi işlemesi için tam **100 ms** zamanı vardır.
- DMA bu sırada ikinci yarıyı doldurmaya devam eder. **200 ms** dolduğunda Tam Aktarım Kesmesi (**Full-Transfer IT**) oluşur.
- CPU, ikinci 200 baytlık veriyi sonraki **100 ms** içinde işlemek zorundadır.
- Eğer CPU'nun veri işleme algoritmaları **100 ms**'den uzun sürerse, DMA döngüsel modda başa dönecek ve işlenmemiş ilk yarının üzerine yazacaktır (**Buffer Overrun**). "DMA kullanıldığı için veri kaçmayacağı" iddiası yanlıştır; yazılımın **100 ms** sınırına uyması zorunludur.

---

6. Protokol Bazlı DMA Uygulamaları ve Teknik Zorluklar

6.1 UART ve Değişken Boyutlu Mesaj Yönetimi

UART [Universal Asynchronous Receiver/Transmitter - Evrensel Asenkron Alıcı/Verici - Asenkron seri iletişim sağlayan çevre birimi] protokolünde DMA kullanılırken karşılaşılan en büyük zorluk, mesaj uzunluklarının önceden kestirilememesidir.

Sabit bir DMA aktarım sayacı (örneğin 50 bayt) tanımlandığında, eğer gelen paket 20 bayt ise DMA kesmesi tetiklenmeyecek ve veri RAM tamponunda bekleyecektir. Bu sorunu çözmek için **Boşta Hat Algılama (IDLE Line Detection / Idle Event)** mekanizması kullanılır.

- Seri hat üzerinde belirli bir süre (örneğin 1 karakter süresi) veri akışı durduğunda donanım bir IDLE olayı üretir.
- CPU, IDLE kesmesi aldığında kalan DMA sayaç değerini okur ve şu formülle değişken paketin boyutunu tam olarak belirler:

```
Alınan Bayt = Toplam Sayaç - Kalan Sayaç
```

6.2 SPI DMA Aktarımı ve Son Bit Ayrımı

SPI [Serial Peripheral Interface - Seri Çevre Birimi Arabirimi - Tam çift yönlü senkron seri iletişim sağlayan çevre birimi] tam çift yönlü (Full-Duplex) bir protokoldür. Gönderilen her bit için eşzamanlı olarak bir bit alınır. Dolayısıyla SPI DMA tasarımlarında hem **TX [Transmit - Gönderme - Veri iletim hattı veya aktarım yönü]** hem de **RX [Receive - Alma - Veri alım hattı veya aktarım yönü]** kanallarının birlikte yönetilmesi kritik önem taşır.

CS Hattı ve Erken Pasifleştirme Riski

SPI iletişiminde hedef entegreyi seçmek için kullanılan **CS [Chip Select - Yonga Seçimi - Seri iletişimde hedef entegreyi etkinleştiren kontrol hattı]** hattının pasife çekilme zamanlaması çok yaygın bir mühendislik hatasına sebep olur:

DMA TX Transfer Complete Kesmesi ≠ SPI Fiziksel Hat Aktarım Bitişi

1. DMA TX kanalı son baytı SPI TX veri yazmacına (Data Register) yazdığı anda "DMA Transfer Complete" kesmesini üretir.
2. Ancak bu anda bayt henüz SPI çevre biriminin dahili kaydırma yazmacındadır (Shift Register) ve fiziksel hat üzerinde iletilmeye devam etmektedir.
3. Eğer yazılımcı sırf DMA kesmesine güvenerek CS hattını mantıksal '1' seviyesine (pasif) çekerse, SPI hattındaki son bitler iletilmeden kesilir ve veri bozulur.

Çözüm: DMA kesmesi alındıktan sonra yazılım, SPI çevre biriminin durum yazmacındaki **BUSY** (Meşgul) bayrağının temizlendiğini ve **TXE** (TX Empty - İletim Yazmacı Boş) bayrağının set edildiğini doğrulamalı, CS hattını ancak bundan sonra pasifleştirmelidir.

---

7. Bellek Mimarisi, Hizalama (Alignment), Cache ve Senkronizasyon

Hizalama (Alignment) ve Donanımsal Adres Şeması

Mikrodenetleyici mimarilerinde bellek erişimleri belirli adres sınırlarına hizalanmalıdır:

- 16-bit erişimler çift adreslerde (**0x2, 0x4, …**)
- 32-bit erişimler 4'ün katı adreslerde (**0x4, 0x8, …**) yapılmalıdır.

Aşağıdaki metin tabanlı bellek Haritasında 32-bitlik bir kelimenin (word) hizalanmış (**0x20000000**) ve hizalanmamış (**0x20000001**) adreslerdeki fiziksel konumu gösterilmektedir:

```
Hizalanmış 32-bit Erişim (Aligned Access @ 0x20000000):
Adres:    0x20000000   0x20000001   0x20000002   0x20000003
Bayt:    [  Byte 0  ] [  Byte 1  ] [  Byte 2  ] [  Byte 3  ]
Durum:   <------------- Tek Bir 32-bitlik Veri Kelimesi (Tek Bus Çevrimi) ------------->

Hizalanmamış 32-bit Erişim (Misaligned Access @ 0x20000001):
Adres:    0x20000000   0x20000001   0x20000002   0x20000003   0x20000004
Bayt:    [ Bosh/Pasif] [  Byte 0  ] [  Byte 1  ] [  Byte 2  ] [  Byte 3  ]
Durum:                <---------------- 32-bitlik Kelime ---------------->
Hata:    Çift Bus Çevrimi Gerekir veya Donanım Hata Sinyali (HardFault) Tetiklenir!
```

Eğer DMA yapılandırmasında öğe genişliği 32-bit seçilip kaynak/hedef adresi hizalanmamış bir bellek konumuna atanırsa donanım hatası (HardFault) veya veri bozulmaları oluşur.

Önbellek Tutarlılığı (Cache Coherency)

Gelişmiş işlemcilerde (Örn: ARM Cortex-M7) Seviye 1 (L1) Önbellek (Cache) bulunur. Cache kullanıldığında işlemci RAM'deki verileri önbelleğe kopyalar. DMA birimi doğrudan fiziksel SRAM ile iletişim kurduğu için önbellekten habersizdir. Bu durum Önbellek Tutarlılığı (Cache Coherency) sorununa yol açar:

- **Bellekten Çevre Birimine Aktarım (TX - Sending):** CPU veri üzerinde değişiklik yapar ancak bu değişiklik sadece Cache'te kalır, SRAM'e yazılmaz. DMA, SRAM'den bayat veriyi (Stale Data) okuyarak çevre birimine gönderir.

- **Çevre Biriminden Belleğe Aktarım (RX - Receiving):** DMA, gelen yeni veriyi SRAM'e yazar. Ancak CPU veriyi okurken Cache'teki eski kopyayı okur; yeni veriyi göremez.

```
Önbellek (Cache) Operasyonlarının Kullanım Sırası:
- TX Öncesi: Clean (Cache'teki güncel veriyi SRAM'e zorla yaz)
- RX Sonrası: Invalidate (Cache'teki geçersiz kopyayı sil, okumayı SRAM'den yapmaya zorla)
```

Yazılım Senkronizasyon Niteleyicileri ve Klasik Sınav Tuzağı

Öğrencilerin ve yazılımcıların en çok yanıldığı nokta **volatile** anahtar kelimesinin işlevidir:

Klasik Sınav Tuzağı: volatile vs Önbellek Tutarlılığı

volatile niteleyicisi, C derleyicisine ilgili değişkenin dış donanımlar tarafından her an değiştirilebileceğini bildirir ve derleyicinin değişken okuma/yazma işlemlerini eksiltmesini (optimization) engeller. Ancak **volatile anahtar kelimesi Önbellek Tutarlılığı (Cache Coherency) sorununu ÇÖZMEZ! volatile** değişkenler L1 Cache belleğe yine yazılabilir. Fiziksel SRAM ile Cache arasındaki veri senkronizasyonunu sağlamak için donanımsal **Clean** ve **Invalidate** komutları şarttır. Ayrıca **volatile** yarış koşullarını veya çekirdekler arası veri erişim sıralamasını tek başına engelleyemez.

Bellek erişim sıralamasını donanımsal düzeyde garanti etmek için bellek bariyerleri kullanılır:

- **DMB [Data Memory Barrier - Veri Bellek Bariyeri - Bellek erişim talimatlarının sırasını garanti eden mimari bariyer]**
- **DSB [Data Synchronization Barrier - Veri Senkronizasyon Bariyeri - Tamamlanmamış bellek işlemlerinin bitmesini bekleyen bariyer]**
- **ISB [Instruction Synchronization Barrier - Talimat Senkronizasyon Bariyeri - İşlemci komut kuyruğunu temizleyerek yeniden yükleyen bariyer]**

---

8. Gömülü Sistemlerde DMA Güvenliği ve İzolasyon: D-Box Mimarisi

Geleneksel monolitik gömülü yazılımlarda DMA denetleyicileri **MPU [Memory Protection Unit - Bellek Koruma Birimi - Bellek adres alanını bölgelere ayırarak erişim yetkilerini denetleyen donanım birimi]** denetimini tamamen bypass eder. Bir DMA denetleyicisi sistem veri yolunda bus master olduğu için, yetkisiz bir görev DMA yazmaçlarını manipüle ederek MPU sınırlarını aşabilir ve çekirdek (kernel) belleğini veya diğer görev alanlarını okuyup yazabilir.

Threat Model (Tehdit Modeli)

- **Confused Deputy (Kararsız Vekil) Saldırıları:** Yetkisiz bir görevin, DMA denetleyicisini kandırarak kendisinin erişemeyeceği korumalı bir bellek alanından (örneğin şifrelerin bulunduğu çekirdek belleği) veri okutması.
- **Rogue DMA Usage:** Üçüncü taraf hatalı/kötücül yazılımların doğrudan DMA yazmaçlarını değiştirerek GPIO, MPU veya kritik çevre birimi durumlarını bozması.

D-Box Referans Mimarisi ve Rol Ayrıştırması

NDSS 2022'de sunulan **D-Box**, TrustZone veya MMU bulunmayan ARM Cortex-M3/M4 işlemcilerde esnek ve güvenli DMA izolasyonu sağlayan akademik bir mimaridir.

D-Box mimarisinde **Rol Ayrıştırması (Role Separation)** esastır:

1. **User Tasks (Kullanıcı Görevleri):** DMA yazmaçlarına erişim yetkileri yoktur. Sadece *neyin* aktarılacağını (kendi bellek tamponlarını) kontrol edebilirler.
2. **Trusted DMA Task (Güvenilir DMA Görevi):** DMA denetleyicisini yapılandırma yetkisine sahip tek ayrıcalıklı görevdir. Bir kullanıcı görevi DMA işlemi istediğinde, istek D-Box güvenlik ilkesine göre denetlenir ve aktarım Güvenilir DMA Görevi tarafından başlatılır. Kullanıcı görevi ile Güvenilir DMA Görevi arasındaki haberleşme **SVC [Supervisor Call - Denetleyici Çağrısı - Ayrıcalıksız yazılımın çekirdek yetkilerine erişmek için ürettiği yazılımsal kesme]** üzerinden yürütülür.

```
D-Box Rol Ayrışımı:
[ User Task ] --(SVC İsteği / Capability)--> [ D-Box Kernel / DMA Policy ]
                                                        | (Doğrulandıysa)
                                                        v
                                              [ Trusted DMA Task ] --> (DMA Controller
Config)
```

D-Box MPU Bölge Yeniden Tanımlamaları ve Öncelik Hiyerarşisi

D-Box, standart FreeRTOS-MPU (F-MPU) yapılandırmasının aşırı izin veren yapısını düzeltmek için MPU bölgelerini katı bir öncelik sırasıyla yeniden tanımlar:

```
D-Box MPU Bölge Haritası ve Öncelik Hiyerarşisi:
 Bölge (-1) : Background Region (Ayrıcalıklı RW - Varsayılan erişimleri engeller)
 Bölge (0)  : Syscalls Region (Ayrıcalıklı/Ayrıcalıksız R - Geçerli SVC giriş noktaları)
 Bölge (1)  : Task Code Region (Ayrıcalıklı/Ayrıcalıksız R - Görev kod alanı izolasyonu)
 Bölge (2)  : Task Stack Region (Ayrıcalıklı/Ayrıcalıksız RW-XN - Çalıştırılamaz Yığın)
 Bölge (3,4,5): User-Defined Regions (Kullanıcı Tanımlı RAM/Çevre Birimi alanları)
 Bölge (6)  : Kernel Code Region (Ayrıcalıklı R - Çekirdek Kodu)
 Bölge (7)  : Kernel Data Region (Ayrıcalıklı RW - Çekirdek Yığını ve Verileri)
```

MPU Öncelik Kuralı: ARM MPU mimarisinde çakışan bellek bölgelerinde *yüksek numaralı bölgeler her zaman düşük numaralı bölgelerin üzerine geçer (priority)*. Bölgeler 6 ve 7'nin en yüksek önceliğe sahip olması, çekirdek (kernel) belleğinin alt seviye kullanıcı tanımlı bölgeler (3, 4, 5) tarafından yanlışlıkla açık hale getirilmesini veya ihlal edilmesini imkansız kılar.

Capability Tabanlı Güvenlik Modeli ve Kod Örneği

D-Box, erişim kontrol listeleri (ACL) yerine **Capability (Yetenek)** tabanlı bir güvenlik modeli kullanır:

> Capability = Nesne (Çevre Birimi Adresi) + EAR (Extensible Access Rights)

EAR [Extensible Access Rights - Genişletilebilir Erişim Hakları - Çevre birimilerine ve off-chip cihazlara özgü ek güvenlik yetkilendirme parametreleri]: Standart okuma/yazma haklarına ek olarak, SPI üzerindeki off-chip harici sensörlerin adreslerini (Chip Select hatları), I2C köle adreslerini veya ADC kanal dizilimlerini içeren gelişmiş yetkilendirme parametreleridir.

D-Box makalesinden alınan aşağıdaki C yapısı (Listing 1), kapabilite tanımının statik kod içindeki uygulamasını göstermektedir:

```
// D-Box Makalesi Listing 1: C Dili Yapıları ile Kapabilite Tanımı
static const PeripheralPermission txPermission[portTOTAL_NUM_PERMISSIONS] =
{
    {(uint32_t *)SPI1, (eRead | eWrite | eFullDuplex), SS_FRAM},
    {(uint32_t *)I2C2, (eRead | eWrite | eFullDuplex), 0x08},
    {(uint32_t *)ADC1, (eRead),                        (ADC_CHANNEL_0 | ADC_CHANNEL_4)}
};
```

D-Box Performans, Güç ve Güvenlik Kazançları

- **ROP Gadget Azaltımı:** D-Box, sürücüleri kullanıcı alanından çıkarıp izolasyon sağladığı için kod alanındaki **ROP [Return-Oriented Programming - Dönüş Odaklı Programlama - Mevcut kod parçacıklarını kullanarak yetkisiz kod yürütmeyi sağlayan saldırı tekniği]** gadget sayısını F-MPU'ya kıyasla **41 kat düşürerek %2.4 seviyesine (%97.6 azalma)** indirir.
- **İşlemci Yükü:** D-Box ek güvenlik denetimlerine rağmen işlemci yüküne yalnızca **%2** ek yük getirir.
- **Güç Tüketimi:** Yalın sürücü yapısı ve DMA'nın etkin kullanımı sayesinde çevre birimi kıyaslamalarında ortalama **%18.2 güç tasarrufu** sağlar.

Durum Çalışması: Wecon PLC Motor Kontrol Sistemi

D-Box, **PLC [Programmable Logic Controller - Programlanabilir Mantıksal Denetleyici - Endüstriyel otomasyonda süreçleri denetleyen dayanıklı bilgisayar sistemi]** cihazlarında (Wecon LX3VM) denenmiştir. Sistemde **PID [Proportional-Integral-Derivative - Oransal-İntegral-Türevsel - Endüstriyel süreçlerde kapalı döngü kontrol sağlayan algoritma]** motor denetimi ve **PWM [Pulse-Width Modulation - Darbe Genişlik Modülasyonu - Dijital sinyallerle analog güç kontrolü sağlayan teknik]** çıkışları bulunurken, dış dünya ile **SCADA [Supervisory Control And Data Acquisition - Merkezi Denetleme ve Veri Toplama - Endüstriyel tesislerin uzaktan izlenmesini ve yönetilmesini sağlayan sistem]** bağlantısı Modbus RTU protokolüyle sağlanmaktadır.

- Modbus RTU haberleşme görevi ve Sensör/PID Motor Kontrol görevi ayrıcalıksız görevler olarak izole edilmiştir.
- Modbus haberleşmesini hedef alan bir uzaktan saldırı gerçekleşse bile, saldırgan D-Box kapabilite sınırlarını aşamadığı için motor kontrol görevine veya DMA üzerinden kritik bellek alanlarına erişememiş, PLC'nin güvenli çalışması korunmuştur.

---

9. Cihaz/Üretici Mimari Karşılaştırması ve Evrensel Kurallar

Farklı mikrodenetleyici üreticilerinin DMA mimarileri evrensel ilkeleri paylaşsa da donanımsal düzeyde belirgin farklılıklara sahiptir.

Donanım Mimarileri Karşılaştırma Tablosu

Parametre	STMicroelectronics STM32L476	Microchip SAM D21	Raspberry Pi RP2040 (*Kaynak Sınırı Notu)
Kanal / Akış Yapısı	2 DMA denetleyicisi, toplam 14 bağımsız kanal.	12 bağımsız DMA kaynağı/kanalı (DMAC).	12 bağımsız DMA kanalı.
Beat / Data Size Desteği	8-bit (Byte), 16-bit (Half-Word), 32-bit (Word).	8-bit, 16-bit, 32-bit beat boyutu.	8-bit, 16-bit, 32-bit veri transferi.
Descriptor (Tanımlayıcı) Yapısı	Yazmaç tabanlı (Registers) konfigürasyon.	SRAM tabanlı Descriptor yapısı (32-bit Next Addr, Src, Dest, 16-bit Count/Control).	Yazmaç ve SRAM tabanlı halka tampon (ring buffer) yapısı.
Tetikleme / Event Sistemi	Çevre birimi kesmeleri ve yazılım tetiklemesi.	Software, Event System ve Çevre birimi tetikleyicileri.	PIO (Programmable I/O), Çevre Birimleri, Timer ve Yazılım.
Güvenlik Özellikleri	Standart MPU entegrasyonu (TrustZone yok).	Standart MPU entegrasyonu (TrustZone yok).	MPU yok, dual-core paylaşımlı veri yolu.

\Kaynak Sınırlaması Uyarısı: Sağlanan kaynak metinlerinde Raspberry Pi RP2040 cihazının detaylı yazmaç seviyesi haritası yer almayıp yalnızca veri kağıdı başlık dizini ve genel donanım özeti bulunduğundan, bu cihaza ait detaylar kaynak sınırları dahilinde sunulmuştur.*

Evrensel İlke

Donanım mimarisi ne olursa olsun evrensel kural şudur: Donanım seviyesinde DMA doğrudan bellek erişimi yaptığı için bellek hizalaması, tampon sahipliği ve senkronizasyon doğrudan yazılımcının ve mimarın sorumluluğundadır.

---

10. Pseudocode ve Kod Uygulama Akışları

1. Memory-to-Memory (M2M) Yapılandırması ve Polling ile Aktarım

```
// STM32 HAL Kütüphanesi Benzeri M2M Polling Örneği
#include "main.h"

DMA_HandleTypeDef hdma_memtomem;
uint8_t Buffer_Src[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
uint8_t Buffer_Dest[10];

void M2M_Polling_Example(void)
{
    // 1. DMA Yapılandırma Parametrelerinin Ayarlanması
    hdma_memtomem.Instance = DMA1_Channel1;
    hdma_memtomem.Init.Direction = DMA_MEMORY_TO_MEMORY;
    hdma_memtomem.Init.PeriphInc = DMA_PINC_ENABLE;      // Kaynak Adresi Artan
    hdma_memtomem.Init.MemInc = DMA_MINC_ENABLE;         // Hedef Adresi Artan
    hdma_memtomem.Init.PeriphDataAlignment = DMA_PDATAALIGN_BYTE; // 8-bit
    hdma_memtomem.Init.MemDataAlignment = DMA_MDATAALIGN_BYTE;    // 8-bit
    hdma_memtomem.Init.Mode = DMA_NORMAL;                // Normal Mod
    hdma_memtomem.Init.Priority = DMA_PRIORITY_LOW;

    // 2. DMA Başlatma
    HAL_DMA_Init(&hdma_memtomem);

    // 3. Aktarımın Başlatılması (Kaynak, Hedef, Öğe Sayısı)
    HAL_DMA_Start(&hdma_memtomem, (uint32_t)Buffer_Src, (uint32_t)Buffer_Dest, 10);

    // 4. Aktarımın Bitişinin Polling (Sorgulama) ile Beklenmesi (100 ms Timeout)
    if (HAL_DMA_PollForTransfer(&hdma_memtomem, HAL_DMA_FULL_TRANSFER, 100) == HAL_OK)
    {
        // Aktarım Başarılı
    }
}
```

2. Kesme Tabanlı (Interrupt-Driven) DMA ve Callback Ataması

Kesme yönetiminde donanımsal kesme denetleyicisi olan **NVIC [Nested Vectored Interrupt Controller - Yuvalanmış Vektörel Kesme Denetleyicisi - ARM Cortex-M mimarisinde kesme önceliklerini ve yönetimini sağlayan donanım birimi]** üzerinden DMA kesme kanalı aktifleştirilir.

```
DMA_HandleTypeDef hdma_memtomem;
uint8_t Buffer_Src[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
uint8_t Buffer_Dest[10];

// Aktarım Tamamlandı Callback Fonksiyonu
void TransferCompleteCallback(DMA_HandleTypeDef *hdma)
{
    // Veri başarıyla kopyalandı, veri işleme adımları
}

void M2M_Interrupt_Example(void)
{
    // ... DMA_Init ayarları yapıldığı varsayılmıştır ...

    // 1. Callback Fonksiyonunun Atanması
```

```
    hdma_memtomem.XferCpltCallback = &TransferCompleteCallback;

    // 2. Kesme Tabanlı DMA Aktarımının Başlatılması (NVIC Kesmesi Aktif)
    HAL_DMA_Start_IT(&hdma_memtomem, (uint32_t)Buffer_Src, (uint32_t)Buffer_Dest, 10);

    // CPU kilitlenmez, diğer işlerine devam eder. Kesme gelince callback çalışır.
}
```

3. İletişim Güvenliği Artırılmış (Thread-Safe) Ping-Pong Tampon Yönetimi

```
#define BUFFER_SIZE 200 // Total 200 eleman (100 Yarı A + 100 Yarı B)
uint16_t PingPongBuffer[BUFFER_SIZE];

typedef enum {
    BUFFER_SECTION_NONE,
    BUFFER_SECTION_HALF_READY,
    BUFFER_SECTION_FULL_READY
} BufferState_t;

// Volatile niteliği ile derleyici optimizasyonu engellenir
volatile BufferState_t buffer_status = BUFFER_SECTION_NONE;

// Half-Transfer Interrupt Servis Rutini (ISR Context)
void HAL_ADC_ConvHalfCpltCallback(void)
{
    buffer_status = BUFFER_SECTION_HALF_READY;
}

// Full-Transfer Interrupt Servis Rutini (ISR Context)
void HAL_ADC_ConvCpltCallback(void)
{
    buffer_status = BUFFER_SECTION_FULL_READY;
}

// Ana Uygulama Döngüsü (Thread Context - Güvenli Durum Yönetimi)
void Process_Loop(void)
{
    while(1)
    {
        BufferState_t current_state;

        // Atomik Durum Okuma ve Kritik Bölüm Koruması (Race Condition Önleme)
        __disable_irq(); // Kesmeler geçici olarak kapatılır
        current_state = buffer_status;
        if (current_state != BUFFER_SECTION_NONE)
        {
            buffer_status = BUFFER_SECTION_NONE; // Bayrak hemen temizlenir
        }
        __enable_irq();  // Kesmeler tekrar açılır

        // Veri İşleme Adımı (Kesmeler Açıkken Yürütülür)
        if (current_state == BUFFER_SECTION_HALF_READY)
        {
            // Sahiplik CPU'da: İlk 100 örneği işle (Ping)
            Process_Data(&PingPongBuffer[0], 100);
        }
        else if (current_state == BUFFER_SECTION_FULL_READY)
        {
            // Sahiplik CPU'da: İkinci 100 örneği işle (Pong)
            Process_Data(&PingPongBuffer[100], 100);
        }
    }
}
```

---

11. Yaygın Hatalar, Sorun Giderme (Troubleshooting) ve Hata Ayıklama

Gömülü sistemlerde DMA geliştirme sürecinde en sık karşılaşılan 5 temel hata ve çözüm adımları şunlardır:

1. **Adres Hizalama ve Veri Genişliği Uyumsuzluğu (Alignment / Width Mismatch):**
   - *Belirti:* Sistem HardFault hatasına geçer veya kopyalanan verilerin üst/alt baytları kaybolur.
   - *Çözüm:* Kaynak ve hedef veri genişliklerini (8/16/32 bit) eşitleyin. Veri tamponlarını bellek hizalama makroları (örneğin **__attribute__((aligned(4)))**) ile tanımlayın.
2. **Cache Invalidate / Clean Adımlarının Unutulması:**
   - *Belirti:* DMA veriyi RAM'e yazmasına rağmen CPU'nun hep eski veriyi okuması.
   - *Çözüm:* DMA aktarımı başlamadan önce TX tamponuna **Clean** işlemi yapın. DMA aktarımı bittikten sonra RX tamponuna **Invalidate** işlemi uygulayın.
3. **Döngüsel Modda Veri Üstüne Yazma (Buffer Overrun):**
   - *Belirti:* İşlenen sinyalde aşamalı kaymalar, veri paketlerinde bozulma.
   - *Çözüm:* CPU işleme süresini optimize edin. Ping-Pong tampon kapasitesini artırarak CPU işleme penceresini genişletin.
4. **SPI Aktarımında CS (Chip Select) Hattının Erken Kaldırılması:**
   - *Belirti:* SPI iletisinin son 1-2 baytının harici entegre tarafından alınamaması.
   - *Çözüm:* CS hattını pasife çekmek için sadece DMA kesmesine güvenmeyin; SPI donanımının **BUSY** bayrağının temizlendiğini doğrulayın.
5. **MPU / Güvenlik Konfigürasyonunda DMA İzin Hataları:**
   - *Belirti:* DMA transferinin tetiklenmesine rağmen verinin hedefe ulaşmaması veya işlemcinin kilitlenmesi.
   - *Çözüm:* DMA denetleyicisinin köle (slave) yapılandırma arayüzünü ayrıcalıksız görevlere kapatın; D-Box benzeri bir Güvenilir DMA Görevi mimarisi kullanın.

---

12. Akademik Makale İncelemesi ve Metrik Doğrulama

Bu raporda referans verilen **D-Box (NDSS 2022)** makalesi, gömülü sistemlerde DMA izolasyonu konusundaki en kapsamlı çalışmalardan biridir.

Atıf Verileri ve Akademik Doğrulama

OpenAlex kayıtlarına göre D-Box makalesi Ekim 2026 itibarıyla **14 atıf** almıştır.

- **DOI: https://doi.org/10.14722/ndss.2022.24053**

Akademik Yaygınlık ve Değerlendirme Uyarısı

Öğrenciler ve araştırmacılar için önemli pedagojik uyarı: Bir akademik makalenin aldığı atıf sayısı tek başına mutlak bir doğruluk veya kalite kriteri değildir. Atıf sayısı; çalışmanın yaşına, yayınlandığı alanın popülerliğine ve dizinlenme kapsamına bağlıdır. Popülerlik ile teknik doğruluk kavramları karıştırılmamalıdır.

D-Box makalesindeki teknik iddialar (ROP gadget azaltımı, güç tasarrufu, işlemci yükü), doğrudan endüstriyel üretici uygulama notları (ST AN4839, AN4031 vb.) ve deneysel ölçüm verileriyle doğrulanarak rapora dahil edilmiştir.

---

13. Kapsamlı Kısaltmalar ve Kavramlar Sözlüğü

Terim / Kısaltma	Açılımı ve Türkçe Karşılığı	Kapsamlı Açıklama / Teknik Tanım
DMA	Direct Memory Access (Doğrudan Bellek Erişimi)	CPU müdahalesi olmadan bellek ile çevre birimleri arasında veri aktarımı sağlayan donanım birimi.
CPU	Central Processing Unit (Merkezi İşlem Birimi)	Komutları yürüten, hesaplamaları ve sistem denetimini yapan ana işlemci.
RAM	Random Access Memory (Rastgele Erişimli Bellek)	Geçici verilerin saklandığı yüksek hızlı çalışma belleği.
SRAM	Static Random Access Memory (Statik Rastgele Erişimli Bellek)	Mikrodenetleyici içindeki yüksek hızlı sistem çalışma belleği.
ADC	Analog-to-Digital Converter (Analog-Dijital Dönüştürücü)	Analog voltaj sinyallerini sayısal değerlere dönüştüren çevre birimi.
UART	Universal Asynchronous Receiver/Transmitter (Evrensel Asenkron Alıcı/Verici)	Asenkron seri iletişim sağlayan çevre birimi.
SPI	Serial Peripheral Interface (Seri Çevre Birimi Arabirimi)	Tam çift yönlü (Full-Duplex) senkron seri iletişim protokolü.
MPU	Memory Protection Unit (Bellek Koruma Birimi)	Bellek adres alanlarını bölgelere ayırıp erişim yetkilerini kısıtlayan donanım.
RTOS	Real-Time Operating System (Gerçek Zamanlı İşletim Sistemi)	Görev zamanlamasını kesin süre sınırları içinde yürüten işletim sistemi.
ISR	Interrupt Service Routine (Kesme Hizmet Rutini)	Kesme oluştuğunda CPU tarafından çalıştırılan özel donanım sürücü fonksiyonu.
NVIC	Nested Vectored Interrupt Controller (Yuvalanmış Vektörel Kesme Denetleyicisi)	ARM Cortex-M mimarisinde kesme önceliklerini yöneten donanım.

Terim / Kısaltma	Açılımı ve Türkçe Karşılığı	Kapsamlı Açıklama / Teknik Tanım
SVC	Supervisor Call (Denetleyici Çağrısı)	Ayrıcalıksız yazılımın çekirdek yetkilerine erişmek için ürettiği yazılımsal kesme.
Beat	Bus Access Beat (Veri Yolu Erişim Vuruşu)	DMA'nın veri yolu üzerinde gerçekleştirdiği tek bir veri erişim (8/16/32-bit) adımı.
Burst	Burst Transfer (Kesintisiz Blok Aktarım)	Veri yolunu bırakmadan peş peşe gerçekleştirilen **n** adet beat aktarımı.
Descriptor	Transfer Control Descriptor (Aktarım Tanımlayıcısı)	Aktarım parametrelerini (kaynak, hedef, sayaç vb.) RAM veya yazmaçta tutan veri yapısı.
Circular Mode	Circular Transfer Mode (Döngüsel Aktarım Modu)	Sayaç bitince adresi otomatik başa saran, kesintisiz aktarım modu.
Ping-Pong Buffer	Double Buffer Strategy (Çift Tampon Yönetimi)	Doldurma ve işlemeyi iki ayrı alanda sırayla yürüten bellek stratejisi.
Ownership	Buffer Ownership (Tampon Sahipliği)	Belirli bir anda tamponun kontrolünün CPU'da mı yoksa DMA'da mı olduğunu belirten kural.
Cache Coherency	Cache Coherency (Önbellek Tutarlılığı)	Önbellekteki veriler ile ana bellekteki (RAM) verilerin güncellik uyumu.
Clean	Cache Clean Operation (Önbellek Temizleme)	Önbellekte güncellenen veriyi ana belleğe (RAM) zorla yazma işlemi.
Invalidate	Cache Invalidate Operation (Önbellek Geçersiz Kılma)	Önbellekteki kopyayı silip CPU'yu RAM'den güncel veriyi okumaya zorlama.
Volatile	Volatile Type Qualifier (Uçucu Değişken Niteleyicisi)	Derleyicinin bellek okuma/yazma optimizasyonu yapmasını engelleyen C dili niteleyicisi.
Memory Barrier	Memory Barrier Instructions (Bellek Bariyeri Talimatları)	Komut ve bellek erişim sıralamasını donanımsal olarak garanti eden mimari komutlar (DMB/DSB/ISB).
ROP	Return-Oriented Programming (Dönüş Odaklı Programlama)	Mevcut kod parçacıklarını kullanarak yürütme akışını ele geçiren saldırı tekniği.
EAR	Extensible Access Rights (Genişletilebilir Erişim Hakları)	Çevre birimlerine ve off-chip cihazlara özgü ek güvenlik yetkilendirme parametreleri.
PLC	Programmable Logic Controller (Programlanabilir Mantıksal Denetleyici)	Endüstriyel otomasyon süreçlerini denetleyen dayanıklı bilgisayar sistemi.

Terim / Kısaltma	Açılımı ve Türkçe Karşılığı	Kapsamlı Açıklama / Teknik Tanım
PID	Proportional-Integral-Derivative (Oransal-İntegral-Türevsel)	Endüstriyel süreçlerde kapalı döngü kontrol sağlayan algoritma.
PWM	Pulse-Width Modulation (Darbe Genişlik Modülasyonu)	Dijital sinyallerle analog güç kontrolü sağlayan teknik.
SCADA	Supervisory Control And Data Acquisition (Merkezi Denetleme ve Veri Toplama)	Endüstriyel tesislerin uzaktan izlenmesini ve yönetilmesini sağlayan sistem.

---

14. Değerlendirme ve Kontrol Soruları

Sorular ve Gerekçeli Çözümleri

Soru 1: DMA denetleyicisi gelen bir sıcaklık verisini analiz edip fanın çalıştırılıp çalıştırılmayacağına karar verebilir mi?

- **Cevap: Hayır.** DMA bir karar verme mekanizmasına sahip değildir; sihirli bir donanım değildir. Sadece tanımlanan adresten diğer adrese veri taşır. Karar verme ve uygulama mantığı CPU'ya aittir.

Soru 2: 100 adet 16-bit veri öğesi içeren bir DMA tamponu bellekte toplam kaç bayt yer kaplar?

- **Cevap: 200 Bayt.** Her 16-bit öğe 2 bayttır. **100 × 2 = 200 Bayt**.

Soru 3: ADC veri toplama sisteminde örnek alma frekansını doğrudan DMA mı belirler?

- **Cevap: Hayır.** ADC'nin örnekleme zamanlamasını genellikle bir Timer (Zamanlayıcı) veya ADC'nin kendi dahili tetikleme mekanizması belirler. DMA sadece dönüşüm bittiğinde üretilen istek sinyaliyle veriyi taşır.

Soru 4: DMA'da Circular (Döngüsel) mod kullanmak sistemin RAM kapasitesini artırır mı?

- **Cevap: Hayır.** Döngüsel mod bellek boyutunu artırmaz. Tampon sonuna ulaşıldığında adresi başa sarar. Eğer eski veri okunmadıysa yeni veri eski verinin üzerine yazılır (Overwrite).

Soru 5: Ping-Pong (Çift Tampon) mekanizmasının temel kullanılma amacı nedir?

- **Cevap:** Veri doldurma işlemi ile veri işleme işlemini iki ayrı bellek alanında ayrıştırarak, CPU ve DMA'nın aynı adrese aynı anda erişmesini (Race Condition) engellemektir.

Soru 6: SPI protokolünde DMA TX kesmesi (Transfer Complete) alındığı anda SPI CS (Chip Select) hattı hemen pasif yapılabilir mi? Nedeniyle açıklayınız.

- **Cevap: Hayır.** DMA TX kesmesi verinin SPI TX yazmacına aktarıldığını gösterir. Veri henüz SPI kaydırma yazmacında (Shift Register) ve hatta iletiliyor olabilir. SPI **BUSY** bayrağı kontrol edilmeden CS kaldırılırsa son bitler bozulur.

Soru 7: C dilindeki **volatile** niteleyicisi L1 Cache içeren sistemlerde Önbellek Tutarlılığı (Cache Coherency) sorununu tek başına çözer mi?

- **Cevap: Hayır. volatile** sadece derleyicinin değişken erişimlerini optimize etmesini engeller. Önbellek ile RAM arasındaki fiziksel veri tutarsızlığını çözmek için donanımsal **Clean** / **Invalidate** işlemleri şarttır.

Soru 8: D-Box güvenlik mimarisinde "Rol Ayrıştırması" (Role Separation) ilkesi Confused Deputy saldırısını nasıl engeller?

- **Cevap:** Kullanıcı görevlerinin DMA yazmaçlarını doğrudan değiştirme yetkisi elinden alınır. Kullanıcı görevi sadece neyi aktaracağını belirler, DMA denetleyicisini yapılandırma yetkisi tek bir "Güvenilir DMA Görevi"ne devredilir. Böylece yetkisiz bellek erişimleri engellenir.

Soru 9: DMA kopyalaması sırasında CPU'nun hiç kilitlenmediği ve veri yolunu hiç kullanmadığı iddiası doğru mudur?

- **Cevap: Yanlıştır.** DMA kopyalama yaparken fiziksel veri yolunu (bus) kullanır. Bu sırada CPU da aynı veri yolundan komut okumak isterse Hakemlik (Arbiter) devreye girer ve CPU kısa süreli erişim gecikmelerine uğrayabilir.

Soru 10: Saniyede 1000 örnek alan ve 100 örneklik Ping-Pong tamponu kullanan bir sistemde, CPU'nun yarım tampon kesmesi geldikten sonra veriyi işlemek için maksimum kaç milisaniyesi vardır?

- **Cevap: 100 ms.** Tamponun yarısı (100 örnek) **100 ms**'de dolar. CPU diğer yarı dolana kadar (**100 ms** içinde) ilk yarıdaki veriyi işlemek zorundadır.

Soru 11: DMA ile bellekten çevre birimine (TX) veri gönderirken ve çevre biriminden belleğe (RX) veri alırken sırasıyla hangi Cache operasyonları uygulanmalıdır?

- **Cevap:** TX aktarımı başlatılmadan önce **Clean** işlemi yapılarak Cache'teki güncellenmiş veri SRAM'e yazılmalıdır. RX aktarımı tamamlandıktan sonra ise **Invalidate** işlemi yapılarak Cache'teki eski veri silinmeli ve CPU'nun SRAM'deki yeni veriyi okuması sağlanmalıdır.

Soru 12: ARM Cortex-M MPU mimarisinde çakışan bellek bölgelerinde erişim yetkisi nasıl belirlenir?

- **Cevap:** ARM MPU mimarisinde *yüksek numaralı MPU bölgesi (Region ID), düşük numaralı MPU bölgesinin üzerine geçer (priority)*. Çakışma durumunda en yüksek bölge numarasının yetki kuralları geçerli olur.

Soru 13: Hizalanmamış (Misaligned) bir bellek adresinden (**0x20000001**) 32-bitlik bir DMA aktarımı başlatıldığında sistemde ne tür bir donanımsal sorun ortaya çıkar?

- **Cevap:** Hizalanmamış adres erişimi donanımsal olarak ekstra veri yolu çevrimlerine veya mimariye bağlı olarak bir donanım hatası kesmesine (**HardFault**) sebep olur ve aktarım verisi bozulabilir.

---

15. Kaynakça

- **[S01]** STMicroelectronics, *AN4031: Using the STM32F2, STM32F4 and STM32F7 series DMA controller*, Application Note.
- **[S02]** Microchip Technology, *SAM D21 Direct Memory Access Controller (DMAC) Overview*, Developer Help Documentation.

- **[S03]** Raspberry Pi Ltd., *RP2040 Datasheet: High-performance microcontroller with unique Programmable I/O*, Technical Documentation.
- **[S04]** STMicroelectronics, *AN5543: Guidelines for enhanced SPI communication on STM32 MCUs and MPUs*, Application Note.
- **[S05]** STMicroelectronics Wiki, *Getting started with DMA*, STM32MCU Resource Page.
- **[S06]** STMicroelectronics, *AN4839: Level 1 cache on STM32F7/STM32H7 MCUs*, Application Note.
- **[S07]** Alejandro Mera, Yi Hui Chen, Ruimin Sun, Engin Kirda, Long Lu, *D-Box: DMA-enabled Compartmentalization for Embedded Applications*, Network and Distributed Systems Security (NDSS) Symposium 2022. DOI: https://doi.org/10.14722/ndss.2022.24053 (OpenAlex Atıf Kaydı: 14 Atıf, Ekim 2026).